

PWM Breathing Light

PWM Breathing Light

1. Learning Objectives
 - PWM Principle
 - PWM Basic Parameters
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. PWM Parameter Configuration
 3. Write Program
 4. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Learn the principle of PWM breathing light.
2. Output a PWM signal through TIMG6_C0 to control the LED indicator light, making it brighten from dim to bright and then dim from bright to dim, similar to human breathing.

PWM Principle

PWM stands for Pulse Width Modulation. It is a very effective technique for digitally controlling analog circuits using microprocessor digital outputs. With its advantages of simple control, flexibility, and good dynamic response, it has become the most widely used control method in power electronics technology. Its application areas include measurement, communication, power control and conversion, motor control, servo control, dimming, switching power supplies, and even some audio amplifiers. Therefore, learning PWM has very important practical significance.

Actually, we can understand it this way: PWM is a method of digitally encoding analog signal levels using high-resolution counters. By using a high-resolution counter, the duty cycle of a square wave is modulated to encode a specific analog signal level. The PWM signal remains digital because at any given moment, the full-scale DC supply is either completely on (ON) or completely off (OFF). The voltage or current source is applied to the analog load through a repetitive pulse sequence of on (ON) or off (OFF). When on, the DC supply is connected to the load; when off, the supply is disconnected. As long as the bandwidth is sufficient, any analog value can be encoded using PWM.

PWM Basic Parameters

PWM is pulse width modulation with two very important parameters: frequency and duty cycle.

- **Frequency:** The frequency of PWM is the reciprocal of the entire period, which is the number of times the PWM signal repeats per unit time. It represents how many signal cycles are completed per second. Frequency determines the rate of PWM signal changes. Higher frequency means shorter signal period and more changes.

- **Duty Cycle:** Duty cycle refers to the proportion of high level in one cycle. Usually expressed as a percentage. It reflects the "width" of the PWM signal, which is the ratio of the high-level duration to the entire period time. Duty cycle determines the average output voltage of the signal. Higher duty cycle means higher average output voltage.

2. Hardware Setup

Earlier we introduced that the MSPM0G series has 7 timers in total, which can be divided into 2 types: general-purpose timers (TIMG) and advanced control timers (TIMA). The PWM function is implemented based on the timer. From the user manual, you can learn that MSPM0G3519 has 7 timers. Which timer has PWM functionality needs to be referred to the pin description in the data manual. Each PWM channel corresponds to a pin on the microcontroller. This pin is not uniquely fixed. There may be one or two pins corresponding to the same channel. For example, TIMG_C0 corresponds to PA5 and PA12, meaning both PA5 and PA12 pins can be configured as timer channel 0. When using, you can choose either one for configuration.

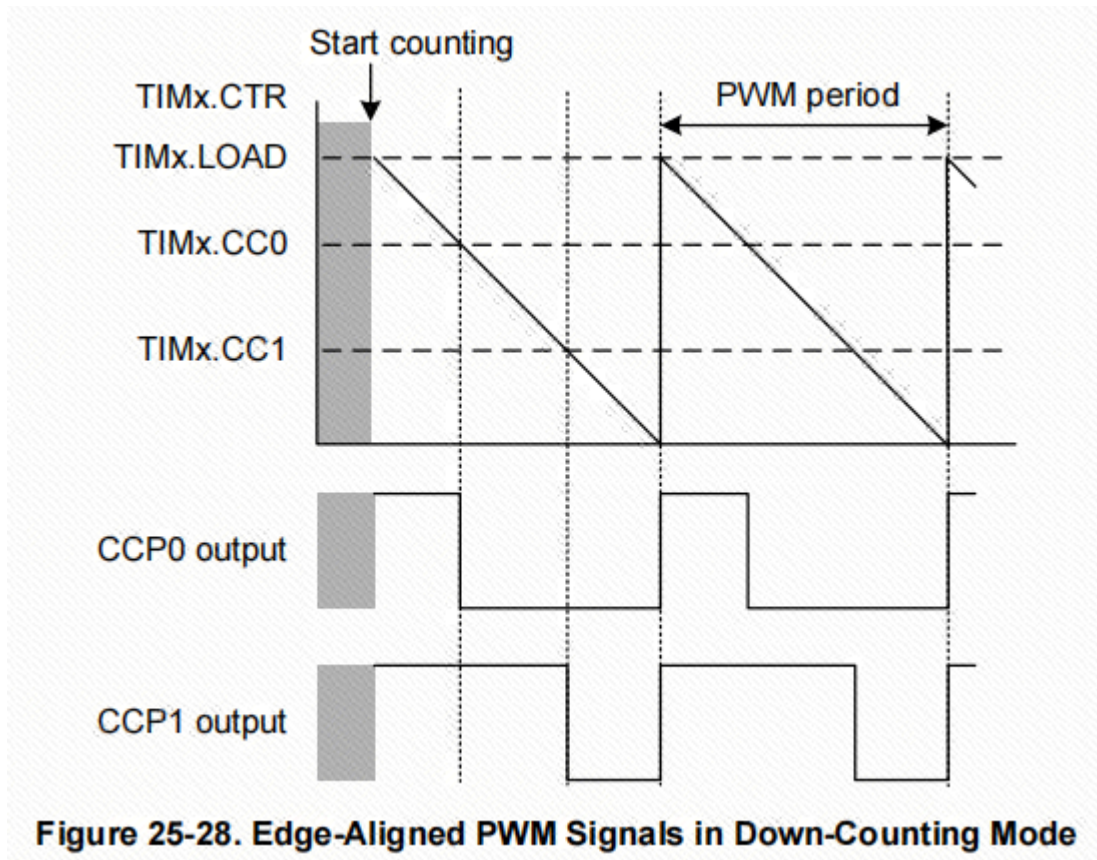
PZ PIN	PIN NAME/ IOMUX REG/ IOMUX ADDR	SIGNAL NAME	IOMUX PF	SIGNAL TYPE	BUFFER TYPE
18	PA5 PINCM10 0x40428024	PA5	1	IO	SDIO (standard)
		TIMG8_C0	2	IO	
		SPI0_PICO	3	IO	
		I2C1_SDA	4	IOD	
		TIMG0_C0	5	IO	
		FCC_IN	6	I	
		TIMG6_C0	7	IO	
		TIMA_FAL1	8	I	
		UART0_CTS	9	I	
		UART4_RTS	10	O	
		UART1_TX	11	O	
		SPI2_CS1	12	IO	
		HFXIN	(Non-IOMUX 1) 0	A	

51	PA12 PINCM34 0x40428084	PA12	1	IO	HSIO (High-speed)
		UART3_CTS	2	I	
		SPI0_SCK	3	IO	
		COMP0_OUT	4	O	
		TIMA0_C3	5	IO	
		FCC_IN	6	I	
		TIMG0_C0	7	IO	
		SPI1_CS1	8	IO	
		SPI0_CS1	9	IO	
		UART7_CTS	10	I	
		UART1_CTS	11	I	
		CAN0_TX	12	O	
		A0_8	(Non-IOMUX 1) 0	A	

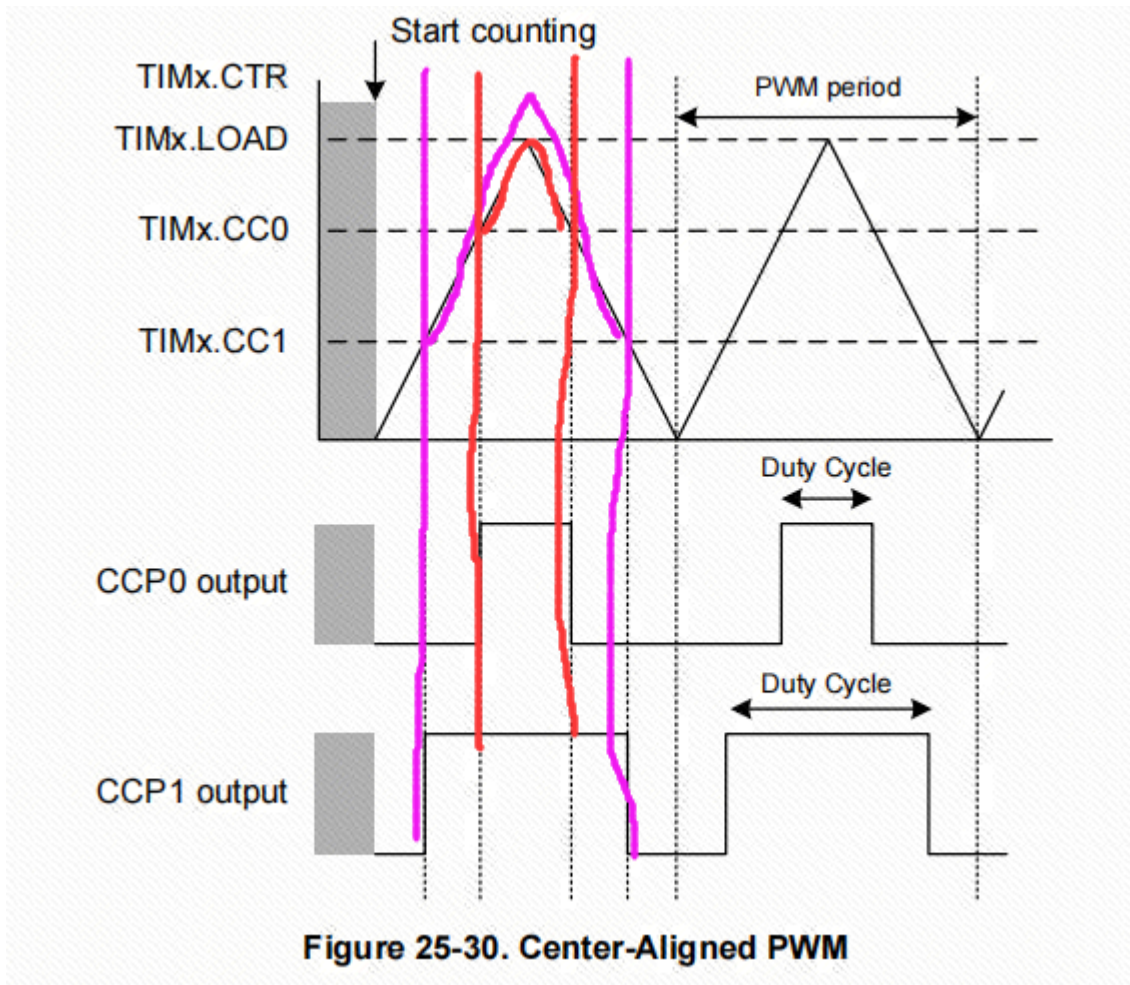
This case uses LED1's PB2 pin, configured as timer 6 channel 0, to generate a PWM signal controlling the light on/off.

23	PB2 PINCM15 0x40428038	PB2	1	IO	SDIO (standard)
		UART3_TX	2	O	
		UART7_CTS	3	I	
		I2C1_SCL	4	IOD	
		TIMA0_C3	5	IO	
		UART1_CTS	6	I	
		TIMG14_C0	7	I	
		UART7_TX	8	O	
		TIMG12_C0	9	IO	
		HFCLK_IN	10	I	
		SPI0_PICO	11	IO	
		TIMA1_C0	12	IO	
		TIMG6_C0	13	IO	

PWM has two modes. One is edge-aligned PWM, which means the timer uses down-counting mode. The output channel outputs high level when the timer count starts. **When the count value equals the output channel's compare value (CC0 and CCP0, CC1 and CCP1 in the figure),** the output channel outputs low level. This continues until the timer count reaches 0, completing one cycle of pulse output.



The other mode is center-aligned mode, which means the timer uses up/down counting mode. The output channel outputs low level when the timer starts. It toggles the level once when the timer counts up and down to the equal compare value of the output channel.

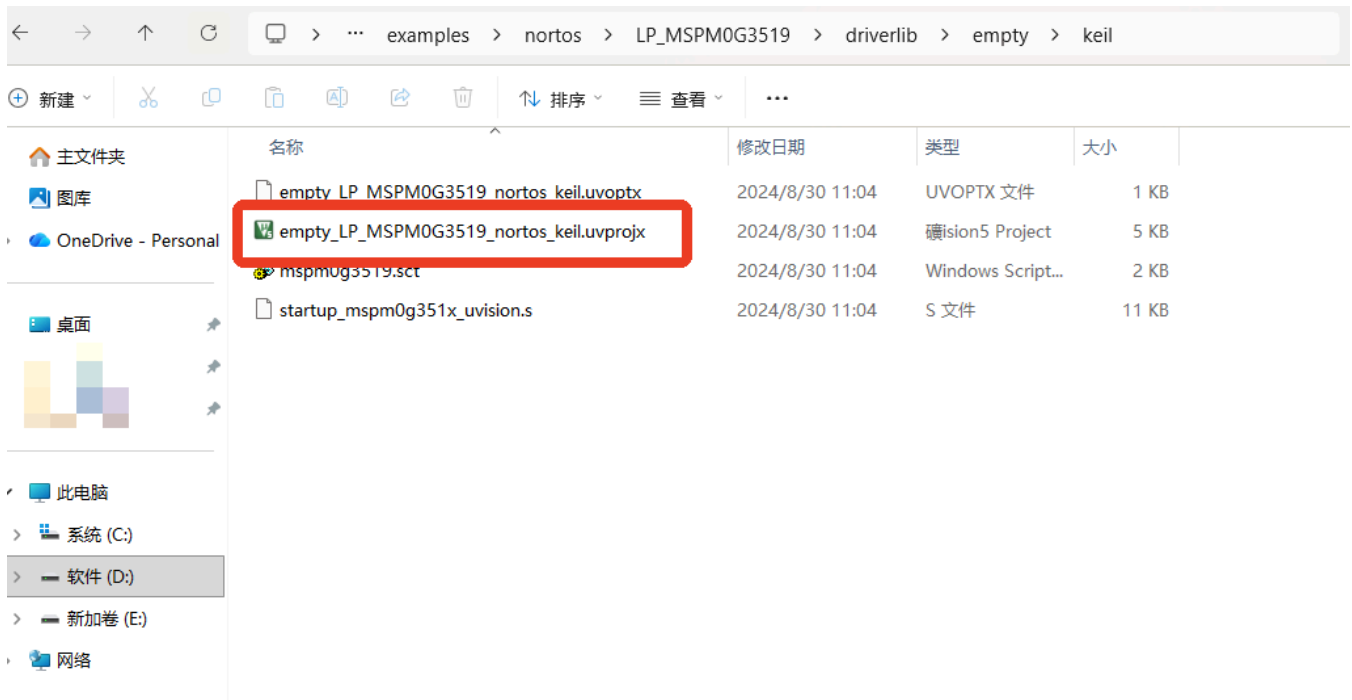


3. Experiment Steps

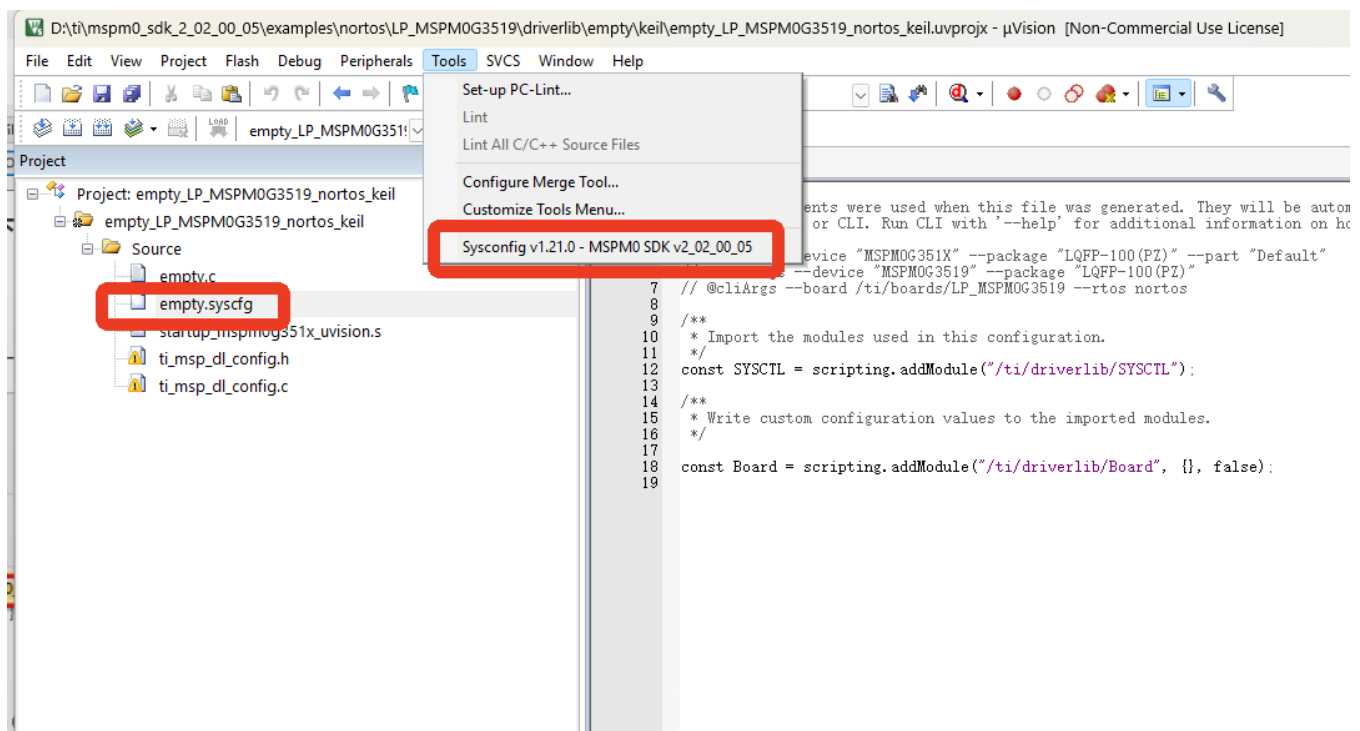
Achieve breathing light effect through driving PWM signal. By changing the PWM signal's duty cycle, you can adjust the current flowing through the LED, thereby achieving LED brightness gradient. Generally, the human eye does not perceive obvious flickering at refresh rates above 80Hz because the flickering cannot be detected at high frequencies. Larger duty cycle means brighter LED; smaller duty cycle means dimmer LED. Therefore, at a fixed frequency, adjusting the duty cycle controls the LED brightness, thereby achieving the breathing light effect. This case will use the onboard LED light (PB2) to achieve breathing light effect, gradually brightening then gradually dimming.

1. Open SYSCONFIG Configuration Tool

Open the empty project from the SDK in KEIL.



After selecting and opening it, open the empty.syscfg file in the KEIL interface. **With the empty.syscfg file open**, then open the SYSCONFIG GUI interface from the Tools menu.



In SYSCONFIG, select MCU peripherals on the left, find the TIMER-PWM option and click to enter. Click ADD in TIMER-PWM to add the PWM peripheral under the timer.

FILE

ABOUT

Type Filter Text...

Software > TIMER - PWM

PROJECT CONFIGURATIO...

Project Config... 1/1

MSPM0 DRIVER LIBRARY ...

SYSTEM (9)

Board 1/1

Configuration NVM

DMA

GPIO 2

MATHACL

RTC

SYSCTL 1/1

SYSTICK 1/1

WWDTC

ANALOG (6)

ADC12

COMP

DAC12

GPAMP

OPA

VREF

COMMUNICATIONS (6)

I2C

I2C - SMBUS

MCAN

SPI

UART 1/4

UART - LIN

TIMERS (6)

TIMER 1/7

TIMER - CAPTURE

TIMER - COMPARE

TIMER - PWM

TIMER - QEI

Timer Fault

TIMER - PWM (0 of 7 Added)

2

ADD

REMOVE ALL

Description

The **Pulse Width Modulation (PWM) Driver module** allows you generate edge-aligned output waveforms.

Under *Basic Configuration* users can:

- Configure clock source
- Configure timer settings
- Configure PWM:
 - Select PWM mode
 - Configure PWM Channels (compare values, duty cycle, etc.)

Under *Advanced Configuration* users can:

- Configure cross trigger

This module supports the different types of Timers included in MSPM0 including TIM0, TIM6, TIM7, and TIM8. The following table includes a description of the features included in MSPM0 including TIM0, TIM6, TIM7, and TIM8. Check the datasheet of your selected device for more detailed information.

TIM Name	Power Domain	Resolution	Prescaler	Repeat	CC Ch	Phase Load
TIM0	PD0	16-bit	8-bit	-	2	-
TIM6	PD1	16-bit	8-bit	-	2	-
TIM7	PD1	16-bit	8-bit	-	2	-
TIM8	PD0	16-bit	8-bit	-	2	-

Click the **Add** button to add a TIMER - PWM to your design

Name PWM_0

Selected Peripheral

Quick Profiles

PWM Profiles Custom

2. PWM Parameter Configuration

Here set channel 0 to 4kHz frequency, 0% duty cycle PWM. Period count value is 1000, compare value is also set to 1000. This way, starting from 1000 and counting down, whenever it encounters 1000, the PWM state toggles once.

PWM_LED

Peripheral does not retain register contents in STOP or STANDBY modes. User should take care to save and restore register configuration in application. See Retention Configuration section for more details.

Name

PWM_LED

Selected Peripheral

TIM6

Quick Profiles

PWM Profiles

Custom

Basic Configuration

Clock Configuration

Timer Clock Source

BUSCLK

Timer Clock Divider

Divided by 8

Calculated Timer Clock Source

4000000

Timer Clock Prescale

1

Calculated Timer Clock Values

Calculated Clock Frequency (Hz)

4000000

Timer Clock Information

61.04 Hz to 2.00 MHz w/ resolution of 250.00 ns

PWM Period Count

1000

Calculated PWM Frequency (Hz)

4000

Start Timer

☒

PWM Configuration

PWM Mode

Edge-aligned Down Counting

PWM Channel(s)

PWM Channel 0

Enable Channel Complimentary Output

None

PWM Channel 0 Channel Specific Configurables for PWM Channel 0

Initial Value

Low

Counter Compare Value

1000

Desired Duty Cycle (%)

0

Actual Duty Cycle (%)

0

Invert Channel

☐

Channel Update Mode

Capture Compare value has immediate effect

Parameter description:

Name: Custom name. Here set to `PWM_LED`

PWM Profiles: Configure PWM template. Default is fine.

Timer Clock Source: Timer clock source. Select `BUSCLK` (bus clock), default frequency is 32MHz.

Timer Clock Divider: Timer clock divider. Set to **8 divider**.

Calculated Timer Clock Source: Shows the clock frequency after division.

Timer Clock Prescale: Timer prescaler. We configured TIMG6, a 16-bit timer, which supports prescaler. No prescaler set here.

Calculated Clock Frequency (Hz): Shows the frequency after division and prescaling.

PWM Period Count: PWM period count value. Here set to 1000.

Calculated Pwm Frequency(Hz): Software automatically calculates PWM frequency, showing 4000.

Start Timer: Whether to start timer. Checked here. Timer PWM output starts on power-up.

PWM Mode: PWM mode. Configured as **Edge-aligned Down Counting** edge-aligned mode with down-counting mode.

PWM Channel(s): PWM channel. Configured to only enable channel 0.

Enable Channel Complimentary Output: Whether to enable channel complementary output. Not enabled here.

Initial Value: Default output state of PWM channel. Configured as default low level output.

Counter Compare Value: PWM compare value. Set to 1000, meaning PWM duty cycle setting range is 0-999.

Desired Duty cycle (%): PWM duty cycle. Default duty cycle 0 is used here.

Actual Duty cycle (%): Shows the actual duty cycle automatically calculated by software.

Invert Channel: Whether to enable inverted output. Not enabled here.

Channel Update Mode: Channel update mode. Configured as **Capture Compare value has immediate effect** - update immediately when compare value is reached.

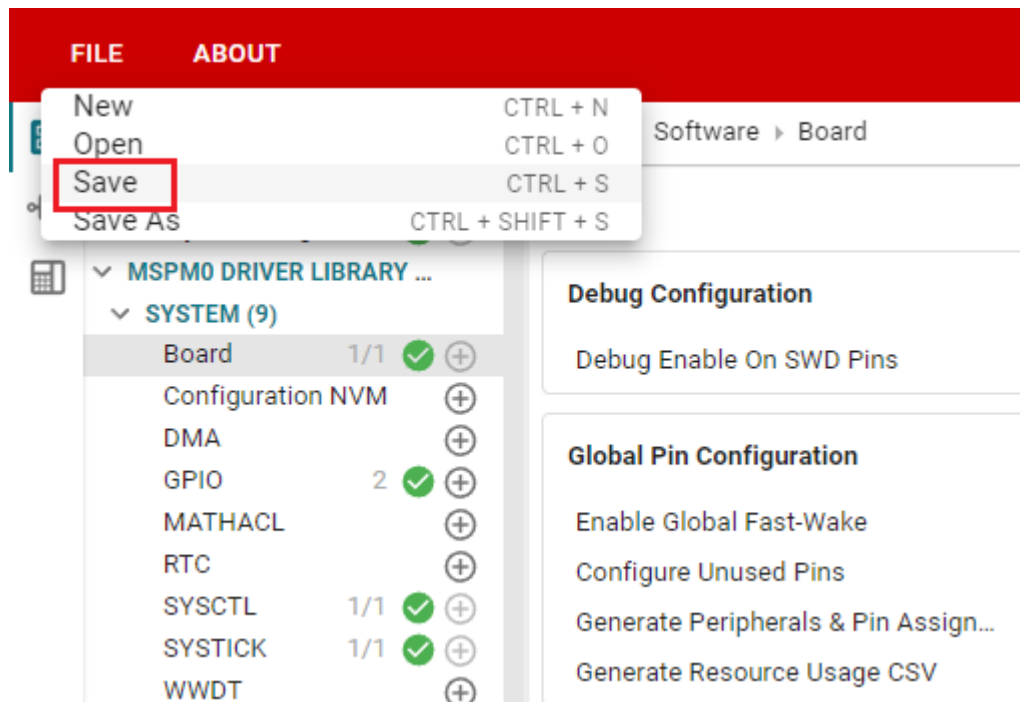
PinMux Peripheral and Pin Configuration		^
Timer Peripheral	TIMG6	▼ 🔒
Capture/Compare Pin 0	PB2/50	▼ 🔒

LED1's control pin is PB2. PB2 has timer 6 channel 0, so select **TIMG6**, **PB2/50** here.

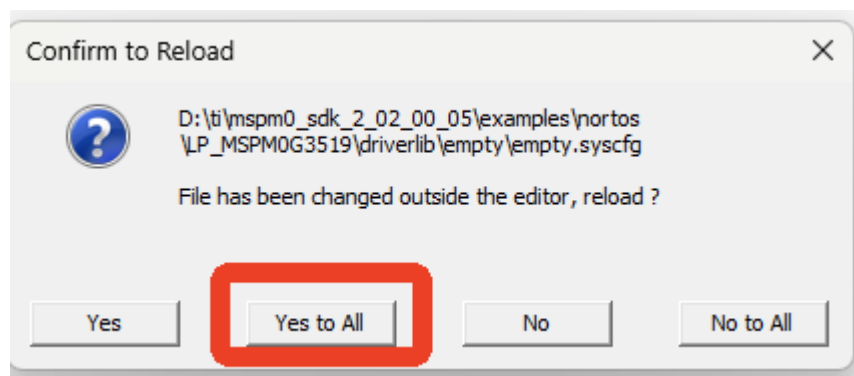
GPIO (1 Added) ⓘ		+ ADD	🗑️ REMOVE ALL
✓ KEY		📄 🗑️	
Name	KEY		
Port	PORTA ▼		
Port Segment	Any ▼		

We have already assigned PB2 of LED1 to PWM. It cannot be used for GPIO peripheral anymore and needs to be removed. Currently, only one button configuration remains in the GPIO peripheral in the example.

Click SAVE to save the SYSCONFIG configuration, then close SYSCONFIG and return to KEIL.



Select **Yes to All** in the popup.



We also need to confirm whether `ti_msp_dl_config.c` and `ti_msp_dl_config.h` files are updated. Simply compile, and the compilation will automatically update to KEIL. If not updated, we can also copy the file content from `SYSCONFIG`.

3. Write Program

In the `empty.c` file, write the following code

```
#include "ti_msp_dl_config.h"

volatile unsigned int delay_times = 0;

//Precise millisecond delay with tick timer
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

int main(void)
{
```

```

int i = 0;

SYSCFG_DL_init();

while (1)
{

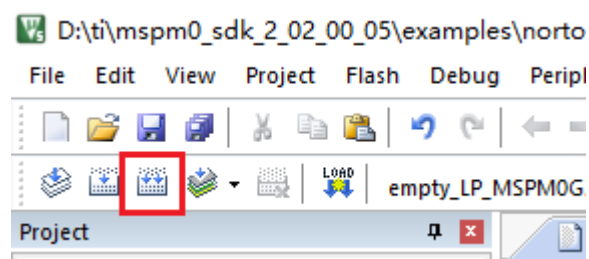
    // Breathing light gradually brightens
    for (i = 0; i <= 999; i++)
    {
        // Set LED Brightness
        DL_TimerG_setCaptureCompareValue(PWM_LED_INST,i,GPIO_PWM_LED_C0_IDX);
        delay_ms(1); // Delay to control the speed of brightness change
    }
    // Breathing light dimming process
    for (i = 999; i > 0; i--)
    {
        // Set LED Brightness
        DL_TimerG_setCaptureCompareValue(PWM_LED_INST,i,GPIO_PWM_LED_C0_IDX);
        delay_ms(1); // Delay to control the speed of brightness change
    }
}

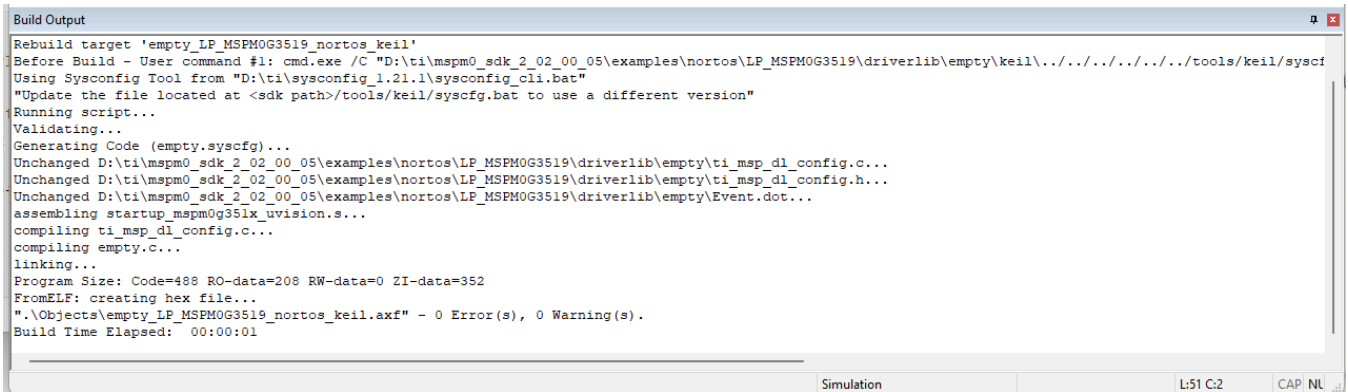
//Tick timer interrupt service function
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

```

4. Compile

Click the Rebuild icon. The following prompt indicates compilation is complete with no errors.





4. Program Analysis

- empty.c

```
int main(void)
{
    int i = 0;

    SYSCFG_DL_init();

    while (1)
    {

        // Breathing light gradually brightens
        for (i = 0; i <= 999; i++)
        {
            // Set LED Brightness
            DL_TimerG_setCaptureCompareValue(PWM_LED_INST, i, GPIO_PWM_LED_C0_IDX);
            delay_ms(1); // Delay to control the speed of brightness change
        }

        // Breathing light dimming process
        for (i = 999; i > 0; i--)
        {
            // Set LED Brightness
            DL_TimerG_setCaptureCompareValue(PWM_LED_INST, i, GPIO_PWM_LED_C0_IDX);
            delay_ms(1); // Delay to control the speed of brightness change
        }
    }
}
```

After initializing the hardware, enter the breathing light loop. Use PWM signal duty cycle to control LED brightness. The breathing light **gradually brightens** and **gradually dims** using two for loops, adjusting the PWM duty cycle from 0 to 999 and then from 999 to 0 progressively. `delay_ms(1)` sets the brightness change interval to 1 millisecond. You can control the breathing light change speed by adjusting the delay function parameter.

5. Experimental Phenomena

After the program is downloaded, you can see the LED light slowly brighten and dim, showing a breathing light effect.

